

ABC458 G - Children Yearn for the Evil Kindergarten 题解

题意概括

一开始场馆里有无限多个小孩，但都没有奖牌。

第 i 天会发生：

1. 收走当前场馆内所有小孩手上的奖牌，总数记为 s ；
2. 把 $s + A_i$ 枚奖牌任意分给当前还在场馆里的小孩；
3. 奖牌数少于 B_i 的小孩退场；其余小孩各自交出 B_i 枚奖牌；
4. 奖牌数至少为 C_i 的小孩可以选择逃脱，也可以留下。

问最终最多能有多少个小孩逃脱。共有 T 组数据。

解题思路

第一步：把问题改写成“固定有多少个小孩要逃脱”

如果最终有 m 个小孩能逃脱，那么这 m 个小孩都必须在第 1 天先活下来。

因为第 1 天结束时：

- 得不到至少 B_1 枚奖牌的小孩会直接退场；
- 退场后就再也回不来了。

所以答案一定不超过 $\left\lfloor \frac{A_1}{B_1} \right\rfloor$ 。

这说明我们只需要关心“前若干个小孩”的边际代价。

第二步：定义后缀 DP

定义 $need_i(m)$ ：

- 表示在第 i 天开始前，如果场馆里有 m 个小孩，
- 那么至少还需要预先持有多少枚奖牌，
- 才能保证这 m 个小孩从第 i 天到第 N 天全部成功逃脱。

如果第 i 天结束后留下 r 个小孩，那么当天有 $m - r$ 个小孩逃脱。

这一天的代价是：

- 所有 m 个小孩先各自至少要有 B_i 枚奖牌，总代价 $m B_i$ ；
- 那些今天逃脱的小孩还要额外各自保留 C_i 枚奖牌，总代价 $(m - r) C_i$ ；
- 留下的 r 个小孩，后面还需要 $need_{i+1}(r)$ 。

再减去当天新增的 A_i 枚奖牌，就得到：

$$need_i(m) = \max\left(0, \min_{0 \leq r \leq m} \left(m B_i + (m - r) C_i + need_{i+1}(r) - A_i \right)\right)$$

直接按这个式子做还是太慢，需要再化简。

第三步：把 $need_i$ 改写成“边际代价序列”

设 w_k 表示从第 $i + 1$ 天开始，让第 k 个小孩最终逃脱所需的**边际代价**。

于是：

$$need_{i+1}(m) = w_1 + w_2 + \dots + w_m$$

并且 $w_1 \leq w_2 \leq \dots$ ，因为让更多小孩逃脱只会越来越难。

现在看第 i 天：

- 如果某个小孩今天直接逃脱，那么它的边际代价是 $B_i + C_i$ ；
- 如果它今天不逃脱，而是留到后面，那么它今天先付出 B_i ，后面再付出对应的 w_k ，总代价是 $B_i + w_k$ 。

所以第 k 个边际代价会变成：

$$u_k = B_i + \min(C_i, w_k)$$

这一步非常关键：

今天直接逃脱，和留到后面，谁便宜就选谁。

第四步：当天新增的 A_i 只会优先抵掉最便宜的边际代价

如果这一天处理前的边际代价序列是：

$$u_1 \leq u_2 \leq \dots$$

那么新增的 A_i 枚奖牌显然应当优先去抵消最便宜的那一部分。

于是新的 $need_i(m)$ 就是：

$$need_i(m) = \max(0, (u_1 + u_2 + \dots + u_m) - A_i)$$

这说明：

1. 先把原序列做一次变换 $u_k = B_i + \min(C_i, w_k)$;
2. 再把 A_i 从序列前端尽量吃掉。

第五步：用分段维护单调序列

我们只需要前 $L = \left\lfloor \frac{A_1}{B_1} \right\rfloor + 1$ 个边际代价：

- 因为答案不会超过 $\left\lfloor \frac{A_1}{B_1} \right\rfloor$;
- 多维护一个位置，只是为了判断“答案是不是已经达到上界”。

整个边际代价序列始终是**非降**的，因此可以用若干段 (value, count) 来维护：

- value：这一段所有边际代价都相等；
- count：这一段长度。

倒着处理每一天时：

1. $B_i + \min(C_i, w_k)$
含义是：整段加上 B_i ，再把过大的后缀截成 $B_i + C_i$ 。
2. 用 A_i 从前往后消耗边际代价。
这会把前缀若干整段清零，至多再把下一段劈成两小段。

因为每次只会在队首和队尾做修改，所以用双端队列就能维护。

最后，边际代价序列最前面的连续 0 的个数，就是可以在不额外预存奖牌的前提下逃脱的小孩数量，也就是答案。

正确性说明

下面分三步证明。

1. need_i(m) 的转移式正确

若第 i 天开始时有 m 个小孩，而第 i 天结束后留下 r 个小孩：

- 每个小孩先至少要有 B_i 枚奖牌，否则会退场，所以必须付出 $m B_i$;
- 其中 $m - r$ 个今天逃脱的小孩，还必须再各自留下 C_i 枚奖牌，所以额外付出 $(m - r) C_i$;
- 留下的 r 个小孩，后缀部分最少还需要 $\text{need}_{i+1}(r)$;
- 当天还能拿到 A_i 枚新增奖牌，所以这部分要减掉。

在所有 r 中取最优，再和 0 取最大值，就得到题解中的转移式。

2. 边际代价转移 $u_k = B_i + \min(C_i, w_k)$ 正确

设后缀边际代价为 $w_1 \leq w_2 \leq \dots$ 。

对于某个小孩，在第 i 天只有两种选择：

- 今天逃脱：边际代价 $B_i + C_i$
- 今天不逃脱、留到后面：边际代价 $B_i + w_k$

显然应选两者中更小的那个，因此新的第 k 个边际代价就是：

$$u_k = B_i + \min(C_i, w_k)$$

并且由于 w_k 本身非降， u_k 仍然非降。

3. 用 A_i 优先抵掉前缀是最优的

边际代价序列已经按从小到大排好。

给定固定的 A_i ，若不优先抵掉最小的边际代价，而去抵掉某个更大的代价，那么把这部分奖牌改去抵掉更小的代价，只会使总所需预存奖牌更少，不会更差。

因此最优方案一定是：

- 从前往后尽量清空最小的边际代价；
- 若剩余奖牌不够清空整段，就只部分减少下一段。

这正是代码中的双端队列维护方式。

综上，算法每一步都保持了 $need_i(m)$ 的真实含义。

最终前缀连续 θ 的个数，恰好就是 $need_1(m) = \theta$ 的最大 m ，也就是最多能逃脱的小孩数。算法正确。

复杂度

设 $L = \left\lfloor \frac{A_1}{B_1} \right\rfloor + 1$ 。

每个测试只维护前 L 个边际代价，并且队列只在两端修改。

- 时间复杂度： $O(N)$ 均摊
- 空间复杂度： $O(L)$

所有测试的 N 之和不超过 3×10^5 ，可以通过。

参考实现

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
```

```
const ll INF = 4000000000000000000LL;

struct Seg{
    ll val;
    ll cnt;
};

void merge_front(deque<Seg> &dq, ll val, ll cnt){
    if(cnt == 0){
        return;
    }
    if(!dq.empty() && dq.front().val == val){
        dq.front().cnt += cnt;
    } else {
        dq.push_front({val, cnt});
    }
}

void merge_back(deque<Seg> &dq, ll val, ll cnt){
    if(cnt == 0){
        return;
    }
    if(!dq.empty() && dq.back().val == val){
        dq.back().cnt += cnt;
    } else {
        dq.push_back({val, cnt});
    }
}

int main(){

    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int T;
    cin >> T;

    while(T--){
        int n;
        cin >> n;
```

```

vector<ll> a(n + 1), b(n + 1), c(n + 1);
for(int i=1; i<=n; i++){
    cin >> a[i] >> b[i] >> c[i];
}

ll limit = a[1] / b[1];
ll len = limit + 1;

deque<Seg> dq;
dq.push_back({INF, len});

ll lazy = 0;

for(int i=n; i>=1; i--){
    // 先做  $u_k = b_i + \min(c_i, w_k)$ 
    lazy += b[i];
    ll cut = b[i] + c[i] - lazy;

    ll tail_cnt = 0;
    while(!dq.empty() && dq.back().val > cut){
        tail_cnt += dq.back().cnt;
        dq.pop_back();
    }
    merge_back(dq, cut, tail_cnt);

    // 再把当天新增的  $a_i$  从最小的边际代价开始抵掉
    ll rem = a[i];
    ll zero_val = -lazy;
    ll zero_cnt = 0;

    while(rem > 0 && !dq.empty()){
        Seg cur = dq.front();
        dq.pop_front();

        ll real_val = cur.val + lazy;
        if(real_val == 0){
            zero_cnt += cur.cnt;
            continue;
        }

        __int128 total = (__int128)real_val * cur.cnt;

```

```

        if(total <= rem){
            rem -= (ll)total;
            zero_cnt += cur.cnt;
            continue;
        }

        ll whole = rem / real_val;
        ll rest = rem % real_val;
        zero_cnt += whole;

        ll left = cur.cnt - whole;
        if(rest == 0){
            merge_front(dq, cur.val, left);
        } else {
            if(left - 1 > 0){
                merge_front(dq, cur.val, left - 1);
            }
            merge_front(dq, cur.val - rest, 1);
        }

        rem = 0;
        break;
    }

    merge_front(dq, zero_val, zero_cnt);
}

ll ans = 0;
for(const auto &seg : dq){
    if(seg.val + lazy != 0){
        break;
    }
    ans += seg.cnt;
}

cout << min(ans, limit) << '\n';
}

return 0;
}

```